

Swami Sahajanand College of Computer Science

B.C.A. SEM - VI

Subject: Web Application Development Using ASP.Net

UNIT 1

Introduction

- ◆ Introduction to ASP.NET.
- ◆ IDE (Integrated Development Environment).
- ◆ Web Configuration File (Web.config) .Config & Global Application Class File (Global.asax).
- ◆ State Management – Session & Cookies.
- ◆ C#.Net Programming Language – Variable, Data type, Operator, Conditional Statement, Looping Statement, Array.
- ◆ Basic Control – Label, Textbox, Button, Radio button, Checkbox, Drop down list

Introduction of Visual Web Developer

❏ Introduction of Visual Web Developer:

- ❏ Visual Web Developer is a type of software which is used to develop website using graphical interface. It is allow to programmer to develop web application by clicking, drag and drop and need to write minimum code.
- ❏ Visual Web Developer is developed specially for building ASP.NET web sites, and as such, it includes lots of tools that will help you in quickly creating ASP.NET web applications.
- ❏ Visual Web Developer available with two editions - standalone and free version as Microsoft Visual Web Developer 2010 and as part of the larger development suite called Visual Studio 2010, which is also available in different versions.
- ❏ Express version of Visual Web Developer is free, it contains all the features and tools you need to create complex and feature-rich web applications.
- ❏ Visual Web Developer is a free web development tool, which allows developers to evaluate the web development and editing capabilities of the other Visual Studio versions at no charge. Its main function is to create ASP.NET websites.
- ❏ Visual Web Developer offers you the following features:
 - ❏ **Web page designs** is a powerful Web page editor that includes WYSIWYG editing and an HTML editing mode with IntelliSense and validation.
 - ❏ **Page design** features Consistent site layout with master pages and consistent page appearance with themes and skins.
 - ❏ **Code editing** A code editor that enables you to write code for your dynamic Web pages in Visual Basic or C#. The code editor includes syntax coloration and IntelliSense.
 - ❏ **Debugging** A debugger that helps you find errors in your programs.
 - ❏ **Controls** An extensive suite of ASP.NET Web server controls that incorporate much of the functionality you need for creating Web sites.
 - ❏ **Data access** Support for displaying and editing data in your Web pages. The data can be in a variety of data stores, including databases or XML files. In many cases, you can add data display and editing to your Web pages without writing any code.
 - ❏ **Security**, personalization, and more Built-in application services that enable you to add membership for login security to your site, profile properties that enable you to maintain user-specific information, and other features, most without requiring any code.
 - ❏ **Development** for hosted sites Tools for publishing sites to your hosting site, including a local Web server for testing.

Q.-1 What is ASP.Net? Also explain Features of Asp.Net in detail.

- ❏ ASP.NET is a Web application framework developed and marketed by Microsoft.
- ❏ It allows programmers to build dynamic Web sites, Web applications and Web services.
- ❏ It was first released in January 2002 with version 1.0 of the .NET Framework.
- ❏ ASP.NET is built on the Common Language Runtime (CLR), allowing programmers to write ASP.NET code using any supported .NET language.
- ❏ ASP.NET takes an object-oriented programming approach to Web page execution.
- ❏ Every element in an ASP.NET page is treated as an object and run on the server thus it is known as server side scripting language.
- ❏ ASP.NET creates Web pages and web-sites HTML, CSS, JavaScript and server scripting.

How ASP.net Works?

- ❏ Basically an ASP.NET page is just the same as an HTML page.

- ❑ An HTML page has the extension .htm. If a browser requests an HTML page from the server, the server sends the page to the browser without any modifications.
- ❑ An ASP.NET page has the extension .aspx. If a browser requests an ASP.NET page, the server processes any executable code in the page, before the result is sent back to the browser.
- ❑ The ASP.NET page above does not contain any executable code, so nothing is executed. In the next examples we will add some executable code to the page to demonstrate the difference between static HTML pages and dynamic ASP pages.

ASP.Net Built-in Objects:

- ❑ ASP.Net has some built-in objects that run on a web server. These objects have methods, properties and collections that are used in application development.
- ❑ The following table lists the ASP.Net built-in objects with a brief description:

Object	Description
Application	Describes the methods, properties, and collections of the object that stores information related to the entire Web application, including variables and objects that exist for the lifetime of the application. You use this object to store and retrieve information to be shared among all users of an application. For example, you can use an Application object to create an e-commerce page.
Request	Describes the methods, properties, and collections of the object that stores information related to the HTTP request. This includes forms, cookies, server variables, and certificate data. You use this object to access the information sent in a request from a browser to the server. For example, you can use a Request object to access information entered by a user in an HTML form.
Response	Describes the methods, properties, and collections of the object that stores information related to the server's response. This includes displaying content, manipulating headers, setting locales, and redirecting requests. You use this object to send information to the browser. For example, you use a Response object to send output from your scripts to a browser.
Server	Describes the methods and properties of the object that provides methods for various server tasks. With these methods you can execute code, get error conditions, encode text strings, create objects for use by the Web page, and map physical paths. You use this object to access various utility functions on the server. For example, you may use the Server object to set a time out for a script.
Session	Describes the methods, properties, and collections of the object that stores information related to the user's session, including variables and objects that exist for the lifetime of the session. You use this object to store and retrieve information about particular user sessions. For example, you can use Session object to keep information about the user and his preference and keep track of pending operations.

Features of ASP.Net**Output Caching:**

- ❑ Since ASP.NET, output caching has is being used to store the generated output of pages and controls.
- ❑ On subsequent requests ASP.NET can serve the HTML content more quickly from memory instead of generating the output from scratch.
- ❑ However this approach has the limitation that the generated output always has to be stored in memory and if the website experiences heavy traffic the memory requirements can be drastic. ASP.NET improves output caching by providing output-cache providers.
- ❑ Output-cache providers can use any storage medium to store the generated output. These storage medium can vary from local or remote disks to cache engines.

Redirecting page to a new link

- ❑ The website content is moved during the lifetime of the web application from one URL to other. This was handled by using the Redirect method to redirect to the new URL.
- ❑ One disadvantage with this method is that the Redirect method results in extra roundtrip to the server.

Enabling Viewstate on Control basis.

- ❑ In the previous versions of ASP.NET we can't turn off the viewstate for entire page and then unable it for some of the controls that need it. We can do so in ASP.NET.
- ❑ This is more convenient than disabling the viewstate on control by basis.You can set the ViewStateMode for the page to false and then enable it for individual controls.

Row Selection in GridView and ListView

- ❑ In previous versions of ASP.NET in Gridview and Listviewrowselection was on the basis of RowIndex. If we selected nth row in a page and moved to other page then nth row was selected on the moved page as well.

Session State Compression

- ❑ There are two options to store session state data out of process one is session state provider that stores data in session state server and second is session state provider that stores data in a Microsoft SQL Server database.
- ❑ The size of the serialized data stored in the session state server and SQL Server can be too large. In ASP.NET 4 there is compression option to store the serialized data. This option can be enabled using compressionEnabled attribute in Session State element.

ClientIDMode Property for generated ClientID

- ❑ Every asp.net control is rendered as an HTML element and a corresponding ClientId is generated for it.
- ❑ If we want to reference the generated HTML element in JavaScript we must know the Id attribute of the generated element.
- ❑ The ID attribute in HTML that is rendered for Web server controls is generated based on the ClientID property of the control.

ListView Control

- ❑ Until ASP.NET 3.5 ListView control required ItemTemplate and LayoutTemplate.
- ❑ ItemTemplate specifies the markup for each item bound to the ListView while LayoutTemplate defines just the enclosing markup for the items in the List View.

Asp.net Directories:

- ❑ In general, the ASP.NET directory structure can be determined by the developer's preferences.
- ❑ Apart from a few reserved directory names, the site can span any number of directories. The structure is typically reflected directly in the URLs.
- ❑ Although ASP.NET provides means for intercepting the request at any point during processing, the developer is not forced to funnel requests through a central application or front controller.
- ❑ The special directory names (from ASP.NET 2.0 on) are:

App_Code

- ❑ This is the "raw code" directory. The ASP.NET server automatically compiles files (and subdirectories) in this folder into an assembly which is accessible in the code of every page of the site. App_Code will typically be used for data access abstraction code, model code and business code. Also any site-specific http handlers and modules and Web service implementation go in this directory. As an alternative to using App_Code the developer may opt to provide a separate assembly with precompiled code.

App_Data

- ❑ The App_Data ASP.NET Directory is the default directory for any database used by the ASP.NET Website. These databases might include Access (mdb) files or SQL Server (mdf) files. The App_Data is the only directory with Write Access enabled for the ASP.NET web application.:

App_LocalResources

- ❑ E.g. a file called CheckOut.aspx.fr-FR.resx holds localized resources for the French version of the CheckOut.aspx page. When the UI culture is set to French, ASP.NET will automatically find and use this file for localization.

App_GlobalResources

- ❑ Holds resx files with localized resources available to every page of the site. This is where the ASP.NET developer will typically store localized messages etc. which are used on more than one page.

App_Themes

- ❑ Adds a folder that holds files related to themes which is a new ASP.NET feature that helps ensure a consistent appearance throughout a Web site and makes it easier to change the Web site's appearance when necessary.

App_WebReferences

- ❑ holds discovery files and WSDL files for references to Web services to be consumed in the site.

Bin

- ❑ Contains compiled code (.dll files) for controls, components, or other code that you want to reference in your application. Any classes represented by code in the Bin folder are automatically referenced in your application.

Asp.Net Directives:

- ❑ Directives specify settings that are used by the page and user-control compilers when the compilers process ASP.NET Web Forms pages (.aspx files) and user control (.ascx) files.
- ❑ ASP.NET treats any directive block (<%@ %>) that does not contain an explicit directive name as an @ Pagedirective (for a page) or as an @ Control directive (for a user control).
- ❑ For syntax information and descriptions of the attributes that are available for each directive, use the links that are listed below.

@ Assembly

- ❑ Links an assembly to the current page or user control declaratively.

@ Control

- ❑ Defines control-specific attributes used by the ASP.NET page parser and compiler and can be included only in .ascx files (user controls).

@ Implements

- ❑ Indicates that a page or user control implements a specified .NET Framework interface declaratively.

@ Import

- ❑ Imports a namespace into a page or user control explicitly.

@ Master

- ❑ Identifies a page as a master page and defines attributes used by the ASP.NET page parser and compiler and can be included only in .master files.

@ MasterType

- ❑ Defines the class or virtual path used to type the Master property of a page.

@ OutputCache

- ❑ Controls the output caching policies of a page or user control declaratively.

@ Page

- ❑ Defines page-specific attributes used by the ASP.NET page parser and compiler and can be included only in .aspx files.

@ PreviousPageType

- ❑ Creates a strongly typed reference to the source page from the target of a cross-page posting.

@ Reference

- ❑ Links a page, user control, or COM control to the current page or user control declaratively.

@ Register

- ❑ Associates aliases with namespaces and classes, which allow user controls and custom server controls to be rendered when included in a requested page or user control.

Advantages of ASP.NET

- ❑ ASP.NET drastically reduces the amount of code required to build large applications.
- ❑ With built-in Windows authentication and per-application configuration, your applications are safe and secured.
- ❑ It provides better performance by taking advantage of early binding, just-in-time compilation, native optimization, and caching services right out of the box.
- ❑ The ASP.NET framework is complemented by a rich toolbox and designer in the Visual Studio integrated development environment. WYSIWYG editing, drag-and-drop server controls, and automatic deployment are just a few of the features this powerful tool provides.
- ❑ Provides simplicity as ASP.NET makes it easy to perform common tasks, from simple form submission and client authentication to deployment and site configuration.
- ❑ The source code and HTML are together therefore ASP.NET pages are easy to maintain and write. Also the source code is executed on the server. This provides a lot of power and flexibility to the web pages.
- ❑ All the processes are closely monitored and managed by the ASP.NET runtime, so that if process is dead, a new process can be created in its place, which helps keep your application constantly available to handle requests.
- ❑ It is purely server-side technology so, ASP.NET code executes on the server before it is sent to the browser.
- ❑ Being language-independent, it allows you to choose the language that best applies to your application or partition your application across many languages.

- ❑ ASP.NET makes for easy deployment. There is no need to register components because the configuration information is built-in.
- ❑ The Web server continuously monitors the pages, components and applications running on it. If it notices any memory leaks, infinite loops, other illegal activities, it immediately destroys those activities and restarts itself.
- ❑ Easily works with ADO.NET using data-binding and page formatting features. It is an application which runs faster and counters large volumes of users without having performance problems

Disadvantages of ASP.Net

- ❑ Does not allow for easy unit tests - the framework tends not to support automatic unit testing with tools like NUnit very well, which makes test-driven development difficult. MVC based frameworks like Struts or ASP.NET MVC do a better job of this.
- ❑ View state - often times viewstate can get really large or have negative effects on performance. This is especially true for some of the more complex server controls.
- ❑ Abstracts the webiness away from web programming - some people feel that ASP.NET does not fit the general architecture of internet and web based applications. They feel like Microsoft tried to follow the Windows Forms desktop paradigm too closely and it makes for developers that don't have a good grasp on how web apps are different than desktop apps.
- ❑ Clicking the browser's "back" button may not return the user to an earlier state of the Ajax-enabled page.
- ❑ Dynamic web page updates also caused some troubles for a user to bookmark a particular state of the application.
- ❑ Ajax opens up another attack vector for malicious code that web developers might not expected for.
- ❑ Any user whose browser does not support Ajax or JavaScript, or simply has JavaScript disabled, will not be able to use its functionality.
- ❑ Platform-specific (only Windows).
- ❑ Complex to use.

Q-2. Explain Web Configuration file (web. Config) in detail.

- ❑ Web config file is used to configure all settings of the web applications.
- ❑ This is very important file of the asp.net web application.
- ❑ The main purpose of the web config file is to provide all necessary information and security to the particular web application.
- ❑ The information and security to the application is provided by different tags.
- ❑ For e.g. connection string tag provides the connection string of the database.
- ❑ There can be more than one connection string of the web application when application uses more than one database.
- ❑ Web.config file is the main setting and configuration file for asp.net web application.
- ❑ It is an xml document that resides in the root directory of the site and contains data about how the web application will act. This information controls module loading, security configuration, session state configuration and application language and compilation settings.
- ❑ Web.config file can also contain application specific items such as database connection strings.
- ❑ The behavior of an asp.net application is affected by different settings in the configuration files: machine.config and web.config.
- ❑ The machine.config file contains default and the machine specific value for all supported settings.
- ❑ The machine settings are controlled by the system administrator and applications are generally not given access to this file.
- ❑ An application however can override the default values by creating web.config files in its roots folder.
- ❑ The web.config file is a subset of the machine.config file.
- ❑ If the application contrians child directories it can define a web.config file for each folder. Scope of each configuration file is determined in a hierarchical top down manner.
- ❑ Any web.config file can locally extend, restrict or override any settings defined on the upper level.
- ❑ Visual studio generates a default web.config file for each project. An application can execute without a web.config file however you cannot debug an application without a web.config file.

Q-3. Explain Global applications class (global.asax) in detail.

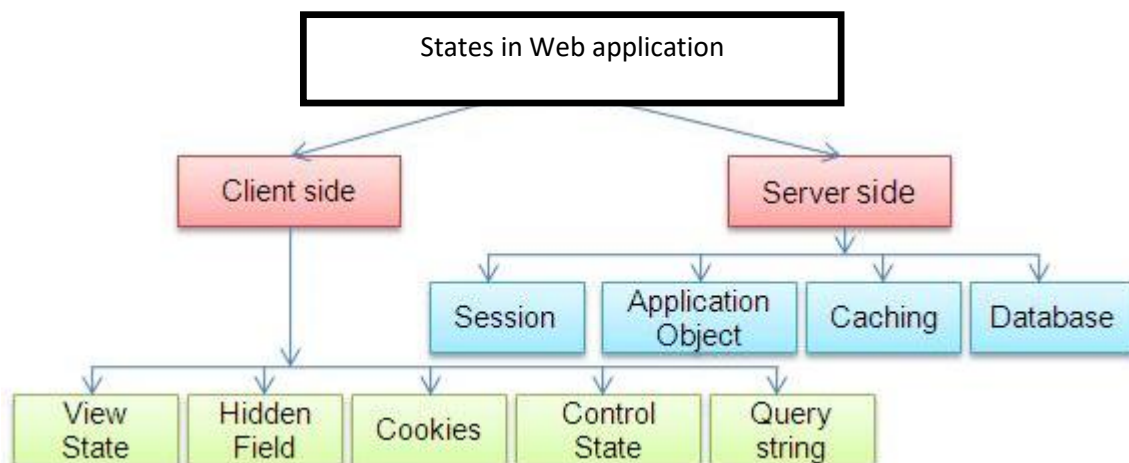
- ❑ The global.asax file also known as the asp.net application file is an optional file that contains code for responding to application level events raised by asp.net or by the http modules.
- ❑ The global.asax file resides in the root directory of an asp.net application.
- ❑ At runtime global.asax is parsed and compiled into dynamically generated .net framework class derived from the httpapplicationbase class.
- ❑ The global.asax file itself is configured so that any direct url request for it is automatically rejected external users cannot download or view the code written within it.
- ❑ The asp.net global.asax file can coexist with the asp global.asax file. You can create a global.asax file either in a WYSIWYG designer, in notepad or as compiled class that you deploy in your application's \bin directory as an assembly. However the latter case you still need a global.asax file that refers to the assembly.
- ❑ The global.asax file is optional. If you do not define the file the asp.net page framework assumes that you have not defined any application or session event handlers.
- ❑ When you save changes to an active global.asax file the asp.net page framework detects that the file has been changed. It completes all current requests for the application sends

the application_onend event to any listeners and restarts the application domain. In effect this reboots the application closing all browser sessions and flushing all state information.

- ❑ When the next incoming request from a browser arrives the asp.net page framework reparses and recompiles the global.asax file and raises the application_onstart event.
- ❑ Events which are not fired for every request :
- ❑ Application_Start() this event is raised when the application starts up and application domain is created.
- ❑ Session_Start() this event is raised for each time a new session begins this is good place to put code that is session specific.
- ❑ Application_Error() this event is raised whenever an unhandled exception occurs in the application. This provides an opportunity to implement generic application wide error handling.
- ❑ Session_End() – this event is called when session of user ends.
- ❑ Application_End() – this event is raised just before when web application ends.
- ❑ Application_Disposed() – this event is fired after the web application is destroyed and this event is used to reclaim the memory it occupies.

Q.-4. What is state? Explain in brief. OR Write a note about State Management.

- ❑ State management means to preserve state of a control, web page, object/data, and user in the application explicitly because all ASP.NET web applications are stateless, i.e., by default, for each page posted to the server, the state of controls is lost. Now a days all web apps demand a high level of state management from control to application level.
- ❑ In traditional web programming, all the information within the page and control gets wiped off on every postback.
- ❑ To overcome this problem, ASP.NET Framework provides various ways to preserve the states at various stages like controlstate, viewstate, cookies, session, etc.
- ❑ These can be defined in client side and server side state management. Please see the image below:



- ❑ State management is an important aspect of any Web application.
- ❑ Because state information is lost between subsequent requests, ASP.NET provides a variety of way to preserve state both server-side and client-side, when your application or controls need to round-trip information across requests.

1) Client Side State:**A) View State**

- ❑ This is used to store request-specific values. ASP.NET provides the server-side notion of a view state for each control.
- ❑ A control can save its internal state between requests using the ViewState property on an instance of the class StateBag.
- ❑ The StateBag class provides a dictionary-like interface to store objects associated with a string key.

B) Hidden field

- ❑ Refers to the control that is not visible when a Web application is viewed in the browser.
- ❑ The content of the control is sent in the HTTP Form collection control along with the values of other controls to the server on page reloads.
- ❑ This control acts as a storage area for any page-specific storing information.

C) Cookies state

- ❑ Client-side cookies are used to store volatile user preferences. Storing cookies on the client is one of the methods that ASP.NET's session state uses to associate requests with sessions.
- ❑ Cookies can also be used directly to persistence data between requests, but the data is then stored on the client and sent to the server with every request.
- ❑ Browsers place limits on the size of a cookie; therefore, only a maximum of 4096 bytes is guaranteed to be acceptable. When the data is stored on the client, the Page_Load method in the file, checks whether the client has sent a cookie. If not, a new cookie is created and initialized and stored on the client.

D) Control State

- ❑ The control state is a way to save a control's state information when the EnableViewState property is turned off.
- ❑ Unlike ViewState a developer can't turn off control state. The control state can be implemented only in controls that you implement.
- ❑ Control state implementation is easy.
- ❑ First, override the OnInit method of the control and add the call for the Page.RegisterRequiresControlState method with the instance of the control to register.
- ❑ Then, override the LoadControlState and SaveControlState in order to save the required state information.

E) Query String

- ❑ The HTTP specification allows for client browsers to pass data as part of the request's query string.
- ❑ This feature is typically used in HTTP GET requests to pass arguments to an application, but you can also use it to pass state information back and forth between the client and server.
- ❑ For example, in the URL
http://www.asp.net/somepage.aspx?userid=12&location=Boston, the query string contains two attribute-value pairs: one named userid and the other named location.
- ❑ Use query strings when you have to store small amounts of state information.
- ❑ The state does not include secret or sensitive information.

- ❑ The state does not provide access to or drive secured parts of your application.
- ❑ Your application runs on a server farm and you have no means of providing a centralized state store.
- ❑ Cookies are disabled.
- ❑ The state must survive across page requests or even across application invocations.
- ❑ Your application will still function if the state is not available.

2) Server Side State:

A) Session State

- ❑ The session state used to store volatile user preferences. To provide individual data for a user during a session, data can be stored with session scope.

B) Application Object State

- ❑ An application state to read a dataset in Application_Start. The `HttpApplicationState` collection used for backward-compatibility with classic ASP and will be familiar to ASP developers. However, the use of static fields in ASP.NET is generally preferred over the use of `HttpApplicationState`.
- ❑ Because an application and all the objects it stores can be concurrently accessed by different threads, it is better to store only infrequently modified data with application scope. Ideally an object is initialized in the `Application_Start` event and further access is read-only.

C) Caching

- ❑ Caching is a technique used to improve the performance and scalability of applications.
- ❑ Caching is most useful in server-based applications, in particular Web applications, but it also applies to many smart client situations.
- ❑ The purpose of caching is to keep state that is expensive to create or retrieve in a more easily accessible form or location.
- ❑ For example, when a user has an active session on your Web application, if you keep a copy of their profile in memory instead of always reading from a remote SQL Server, your application performance improves.

D) Database

- ❑ ASP.NET sessions can also be stored in a SQL Server database.
- ❑ Storing sessions in SQL Server offers resilience that can serve sessions to a large web farm that persists across IIS restarts.
- ❑ SQL based Session state is configured with `aspnet_regsql.exe`. This utility is located in .NET Framework's installed directory
C:\<windows>\microsoft.net\framework\<version>.
- ❑ Running this utility will create a database which will manage the session state.

C#.NET Programming

Q. - 5. What is C#? Explain Variable, Data type and Operator in C# Asp.Net.

- ❑ C# is pronounced "C-Sharp".
- ❑ It is an object-oriented programming language created by Microsoft that runs on the .NET Framework.
- ❑ C# has roots from the C family, and the language is close to other popular languages like
- ❑ The first version was released in year 2002. The latest version,
- ❑ C# is used for:
 - Mobile applications
 - Desktop applications
 - Web applications
 - Web services
 - Web sites
 - Games
 - VR
 - Database applications
 - And much, much more!
- ❑ About C#.Net Programming Language.
- ❑ C# works closely with the .NET ecosystem, which provides the runtime, libraries and tools for application development.
- ❑ Over time, several frameworks and technologies have evolved around C# to support diverse platforms.

What is C#.NET?

- ❑ C#.NET refers to using the **C# language** within the **.NET platform**.
 - **C#** → Programming language
 - **.NET** → Framework/runtime that provides libraries, tools, and execution environment Together, they allow developers to write secure, scalable, and high-performance applications.

Key Features of C#**❑ Simple & Modern**

- ❑ Easy to learn syntax (similar to C++ and Java)
- ❑ Automatic memory management (Garbage Collection)

❑ Object-Oriented

- ❑ Supports core OOP concepts:
- ❑ Class & Object
- ❑ Inheritance
- ❑ Polymorphism
- ❑ Encapsulation
- ❑ Abstraction

❑ Type-Safe

- ❑ Reduces runtime errors
- ❑ Strong data type checking at compile time

❑ Rich Library Support

- ❑ Uses **.NET Class Library**
- ❑ Pre-built functions for file handling, database access, networking, security, etc.

❑ Language Interoperability

- ❑ Can work with other .NET languages like VB.NET

- ❑ **Secure & Robust**
- ❑ Built-in security features
- ❑ Exception handling with try-catch-finally

- ❑ **Applications of C#.NET**
- ❑ C#.NET is widely used for:
- ❑ **Web Applications** – ASP.NET, ASP.NET Core
- ❑ **Desktop Applications** – Windows Forms, WPF
- ❑ **Mobile Applications** – Xamarin, .NET MAUI
- ❑ **Cloud Applications** – Azure-based services
- ❑ **Game Development** – Unity Game Engine
- ❑ **Enterprise Applications** – Banking, ERP systems

Q-6. Explain Variable available in C#.NET.

- ❑ A variable in C# is a named memory location used to store data and its value can change during program execution.
- ❑ Each variable must be declared with a specific data type that defines the kind of values it can hold.
- ❑ Depending on where and how they are declared, ASP.NET applications commonly use several types of variables to manage data across pages, sessions, and requests.

Types of Variables

1. Local variables
2. Instance variables or Non - Static Variables
3. Static Variables or Class Variables
4. Constant Variables
5. Readonly Variables

1. Local Variable

- ❑ A variable defined within a block or method or constructor is called local variable.
- ❑ These variables are created when the lock is entered or the function is called and destroy after exiting from the block or when the call returns from the function.
- ❑ The scope of these variables exists only within the block in which the variable is declared.
- ❑ i.e. we can access these variables only within that block.

❑ Definition:

- ❑ Variables declared **inside a method or block** and used only within that method.

- ❑ **Scope:** Only within the method where they are declared.

❑ Example 1:

```
using System;
protected void Button1_Click(object sender, EventArgs e)
{
    int number = 10; // Local variable
    string name = "Manish";
}
```

Characteristics

- ❑ Created when the method is called
- ❑ Destroyed when the method finishes
- ❑ Cannot be accessed outside the method

2. Instance Variables or Non - Static Variables

- ❑ Instance variables in C# are non-static variables declared inside a class but outside methods or constructors.

- ❑ They are created when an object is instantiated and destroyed when the object is destroyed and can have access specifies.

- ❑ **Definition:**

Variables declared **inside a class but outside methods**.

- ❑ **Scope:** Accessible by all methods of that class.

- ❑ **Example:**

```
using System;
public partial class WebForm1 : System.Web.UI.Page
{
    int count = 0; // Instance variable

    protected void Page_Load(object sender, EventArgs e)
    {
        count++;
    }
}
```

Characteristics

- ❑ Belong to an object of the class
- ❑ Each object gets its own copy

3. Static Variables or Class Variables

- ❑ Static variables in C# are class-level variables declared with the static keyword inside a class but outside methods or constructors.

- ❑ Only one copy exists per class, shared by all objects and they are created at program start and destroyed when execution ends.

- ❑ **Definition:**

- ❑ Variables declared with the **static keyword** and shared across all instances of the class.

- ❑ **Note:** To access static variables, there is no need to create any object of that class, simply access the variable as: *class_name.variable_name*;

- ❑ **Example:**

```
using System;
public class Test
{
    public static int totalUsers = 0;
}
```

Characteristics

- ❑ Shared among all objects
- ❑ Only one copy exists in memory

4. Application Variables

- ❑ **Definition:**

- ❑ Variables stored in the Application object and shared by all users and sessions.

- ❑ **Example:**

```
Application["TotalVisitors"] = 100;
```

- ❑ **Access:**

```
int visitors = (int)Application["TotalVisitors"];
```

Characteristics

- ❑ Global across the entire web application
- ❑ Stored on the server

5. Session Variables**❏ Definition:**

❏ Variables stored in the Session object to maintain user-specific data across pages.

❏ Example:

```
Session["Username"] = "Rahul";
```

Access:

```
string user = Session["Username"].ToString();
```

Characteristics

❏ Unique for each user

❏ Exists until session expires or user logs out

6. ViewState Variables**❏ Definition:**

❏ Used to preserve page and control values between postbacks.

❏ Example:

```
ViewState["Counter"] = 1;
```

Access:

```
int count = (int)ViewState["Counter"];
```

Characteristics

❏ Stored in the page itself (hidden field)

❏ Works only within the same page

7. Query String Variables**❏ Definition:**

❏ Data passed between pages through the URL.

❏ Example URL

```
Page2.aspx?name=Manish
```

Access in ASP.NET

```
string name = Request.QueryString["name"];
```

Characteristics

❏ Visible in URL

❏ Limited size

8. Cookies**❏ Definition:**

❏ Small data stored on the client browser.

❏ Example:

```
Response.Cookies["UserName"].Value = "Rahul";
```

Access:

```
string name = Request.Cookies["UserName"].Value;
```

❏ Characteristics

❏ Stored in user's browser

❏ Can persist after the session

Variable Type	Scope	Lifetime
Local Variable	Inside method	Method execution

Variable Type	Scope	Lifetime
Instance Variable	Class level	Object lifetime
Static Variable	Class shared	Application lifetime
Application Variable	All users	Application running
Session Variable	Per user	Session duration
ViewState	Same page	Page postback
Query String	Between pages	Single request
Cookies	Client browser	Until expiration

Q-7. Explain Data types available in C#.NET.

- ❑ C#, data types define the kind of values a variable can hold. Since C# is strongly typed, every variable or constant must be declared with a specific data type, such as int, char or float.
- ❑ Data Types in C# is Mainly Divided into 3 Categories:
 - **Value Data Types**
 - **Reference Data Types**
 - **Pointer Data Type**
- ❑ **Value Data Types**
- ❑ In C#, the Value Data Types will directly store the variable value in memory and it will also accept both signed and unsigned literals.
- ❑ Value types store the actual value directly in memory.
- ❑ The derived class for these data types are System.ValueType.
- ❑ Following are different Value Data Types in C# programming language.

Data Type	Description	Example
int	Stores whole numbers	int age = 25;
float	Stores decimal numbers (single precision)	float price = 10.5f;
double	Stores large decimal numbers	double pi = 3.14159;
decimal	High-precision decimal (used in money)	decimal amount = 100.50m;
char	Stores a single character	char grade = 'A';
bool	Stores true/false value	bool isLogin = true;
byte	Stores small numbers (0-255)	byte level = 10;
long	Stores very large integers	long population = 9000000000;
short	Stores smaller integer values	short num = 100;

❏ **Example:**

```
int number = 100;
bool isActive = true;
char letter = 'A';
```

❏ **Characteristics**

- ❏ Stored in stack memory
- ❏ Faster access
- ❏ Cannot contain null (unless nullable)

2. Reference Types

- ❏ Reference types store the memory address of the value instead of the actual value.
- ❏ Common Reference Data Types.

Data Type	Description	Example
string	Stores text	string name = "Rahul";
object	Base type for all types	object obj = "Hello";
class	User-defined type	Student s = new Student();
array	Collection of values	int[] numbers = {1,2,3};

❏ **Example:**

```
string name = "Amit";
int[] marks = {80, 90, 75};
```

❏ **Characteristics**

- ❏ Stored in heap memory
- ❏ Can store null values
- ❏ Multiple variables can reference the same object

3. Nullable Types

- ❏ Nullable types allow value types to store null values.

❏ **Syntax:**

```
int? number = null;
```

❏ **Example:**

```
int? age = null;
bool? isApproved = null;
```

❏ **Characteristics**

- ❏ Used when a value may or may not exist
- ❏ Mostly used in database operations

4. User-Defined Types

- ❏ Programmers can create their own data types.
- ❏ Examples include:

Type	Description
class	Blueprint for objects
struct	Similar to class but value type
enum	Set of named constants
interface	Defines methods without implementation

❏ **Example:**
enum Days

```
{
    Monday,
    Tuesday,
    Wednesday
}
```

Category	Examples
Value Types	int, float, double, char, bool, decimal
Reference Types	string, object, class, array
Nullable Types	int?, bool?, double?
User Defined Types	class, struct, enum, interface

Q-8. Explain Operators of Asp.Net C#.

- ❏ In **ASP.NET using C#**, **operators** are symbols used to **perform operations on variables and values** (such as addition, comparison, or logical decisions).
- ❏ Operators in C# are mainly divided into the following types:

1. Arithmetic Operators

- ❏ Used to perform mathematical calculations.

Operator	Description	Example
+	Addition	a + b
-	Subtraction	a - b
*	Multiplication	a * b
/	Division	a / b
%	Modulus (remainder)	a % b

❏ **Example:**

```
int a = 10;
int b = 5;
```

```
int sum = a + b;
int diff = a - b;
int product = a * b;
int division = a / b;
int remainder = a % b;
```

2. Relational (Comparison) Operators

- ❏ Used to compare two values. The result is true or false.

Operator	Description	Example
==	Equal to	a == b
!=	Not equal to	a != b
>	Greater than	a > b
<	Less than	a < b
>=	Greater than or equal	a >= b
<=	Less than or equal	a <= b

❏ Example:

```
int a = 10;
int b = 20;
bool result = a < b; // true
```

3. Logical Operators

❏ Used to combine multiple conditions.

Operator	Description
&&	Logical AND
	Logical OR
!	Logical NOT

❏ Example:

```
int age = 25;

if(age > 18 && age < 60)
{
    Response.Write("Eligible");
}
```

4. Assignment Operators

❏ Used to assign values to variables.

Operator	Example
=	a = 10
+=	a += 5
-=	a -= 3
*=	a *= 2
/=	a /= 4
%=	a %= 3

❏ Example:

```
int a = 10;
a += 5; // a = a + 5
```

5. Increment and Decrement Operators

❏ Used to increase or decrease a value by 1.

Operator	Description
++	Increment
--	Decrement

❏ Example:

```
int a = 5;

a++; // a = 6
a--; // a = 5
```

6. Conditional (Ternary) Operator

❏ Used as a short form of if-else.

❏ **Syntax:**

condition ? value1 : value2;

❏ **Example:**

int age = 20;

string result = (age >= 18) ? "Adult" : "Minor";

7. Bitwise Operators

❏ Used to perform operations on binary numbers.

Operator	Description
&	AND
	OR
^	XOR
~	NOT
<<	Left shift
>>	Right shift

❏ **Example:**

int a = 5;

int b = 3;

int result = a & b;

Operator Type	Examples
Arithmetic	+, -, *, /, %
Relational	==, !=, >, <
Logical	&&, , !
Assignment	=, +=, -=
Increment/Decrement	++, --
Conditional	?:
Bitwise	&, , ^

Q. - 9. Explain Conditional Statements in C# Asp.Net.

- ❑ Conditional statements are used to make decisions in a program based on conditions.
- ❑ These are very important from an examination point of view.

1. if Statement

Use: The if statement executes a block of code when the condition is true.

Syntax:

```
if (Condition)
{
    Statement;
}
```

Example:

```
int num = 10;
if (num > 0)
{
    Console.WriteLine("Number is positive");
}
```

2. if-else Statement

- ❑ **Use:** The if-else statement executes one block if the condition is true and another if it is false.

Syntax:

```
if (Condition)
{
    Statement;
}
else
{
    Statement;
}
```

Example:

```
int age = 16;
if (age >= 18)
{
    Console.WriteLine("Eligible to vote");
}
else
{
    Console.WriteLine("Not eligible to vote");
}
```

3. else-if Ladder

- ❑ **Use:** The else-if ladder is used to test multiple conditions.

Syntax:

```
if (Condition)
{
    Statement;
}
else if (Condition)
{
    Statement;
}
else
{
    Statement;
}
```

```
Statement;  
}
```

Example:

```
int marks = 75;  
if (marks >= 80)  
{  
    Console.WriteLine("Grade A");  
}  
else if (marks >= 60)  
{  
    Console.WriteLine("Grade B");  
}  
else  
{  
    Console.WriteLine("Grade C");  
}
```

4. Switch Statement

Use: The switch statement selects one execution path from multiple choices.

Syntax:

```
switch (Condition)  
{  
    case 1:  
        Statement;  
        break;  
    case 2:  
        Statement;  
        break;  
    case 3:  
        Statement;  
        break;  
    default:  
        Statement;  
        break;  
}
```

Example:

```
int day = 3;  
switch (day)  
{  
    case 1:  
        Console.WriteLine("Monday");  
        break;  
    case 2:  
        Console.WriteLine("Tuesday");  
        break;  
    case 3:  
        Console.WriteLine("Wednesday");  
        break;  
    default:  
        Console.WriteLine("Invalid Day");  
        break;  
}
```

5. Nested if

❏ Use: Nested if statement is used to print an inner condition.

Syntax:

```
If(Condition)
{
    If(Condition)
    {
        Statement;
    }
    else
    {
        Statement;
    }
}
```

Example:

```
a=5;
b=10;
c=7;
If(a>b)
{
    If(a>c)
    {
        Console.WriteLine("A is Big");
    }
    else
    {
        Console.WriteLine("C is Big");
    }
}
else
{
    If(b>c)
    {
        Console.WriteLine("B is Big");
    }
    else
    {
        Console.WriteLine("C is Big");
    }
}
```

Q. - 10. Explain Looping Statements in C# Asp.Net.

- ❑ Looping statements are used to repeat a block of code multiple times.

1. for Loop

- ❑ **Use:** Used when the number of iterations is known.

Syntax:

```
for (Initialization; Condition; Increment/Decrement)
{
    Statement;
}
```

Example:

```
for (int i = 1; i <= 5; i++)
{
    Console.WriteLine(i);
}
```

2. while Loop

- ❑ **Use:** The while loop checks the condition before executing the loop body.

Syntax:

```
while(Condition)
{
    Statement;
    Increment/Decrement;
}
```

Example:

```
int i = 1;
while (i <= 5)
{
    Console.WriteLine(i);
    i++;
}
```

3. do-while Loop

- ❑ The do-while loop executes at least once.

Syntax:

```
do
{
    Statement;
    Increment/Decrement;
} while (Condition)
```

Example:

```
int i = 1;
do
{
    Console.WriteLine(i);
    i++;
} while (i <= 5)
```

4. foreach Loop

- ❑ The foreach loop is used to iterate through arrays or collections.

Syntax:

```
foreach(Variable in Array)
{
    Statement;
}
```

Example:

```
int[] nums = {10, 20, 30};
foreach(int n in nums)
{
    Console.WriteLine(n);
}
```

Array

Q.-11. What is an Array? Explain Array in Asp.Net with C#.

What is an Array?

- ❑ An array is a collection of elements of the same data type, stored in a fixed-size, and ordered structure. Each element is accessed using an index, which starts from 0.
- ❑ An array is a container that holds multiple values of the same type in a single variable.

Definition

- ❑ An array is a fixed-size data structure that stores elements of the same type in continuous memory locations. Each element is accessed using an index number starting from 0.

Example:

Index: 0 1 2 3
Values: 10 20 30 40

Syntax of Array

```
datatype[] arrayname = new datatype[size];
```

Example:

```
int[] numbers = new int[5];
```

Example Program:

```
int[] marks = new int[3];
marks[0] = 80;
marks[1] = 90;
marks[2] = 75;

Response.Write(marks[1]);
```

Output:

90

❏ Types of Array

- ❏ Stores elements in **one row (linear form)**.

1. Single Dimensional Array

- ❏ Stores elements in one row (linear form).

- ❏ Example:

```
int[] numbers = {10, 20, 30, 40};

for(int i = 0; i < numbers.Length; i++)
{
    Response.Write(numbers[i]);
}
```

2. Multidimensional Array

- ❏ Stores data in rows and columns (table format).

- ❏ Example:

```
int[,] marks = new int[2,2];

marks[0,0] = 80;
marks[0,1] = 90;
marks[1,0] = 70;
marks[1,1] = 60;
```

```
Response.Write(marks[0,1]);
```

- ❏ Output: 90

3. Jagged Array

- ❏ An array of arrays where each row can have different sizes.

- ❏ Example:

```
int[][] numbers = new int[2][];
numbers[0] = new int[] {1,2,3};
numbers[1] = new int[] {4,5};
```

```
Response.Write(numbers[0][2]);
```

- ❏ Output: 3

- ❏ Important Properties of Arrays

Property	Description
Length	Returns number of elements
Rank	Returns number of dimensions
GetLength()	Returns size of a specific dimension

- ❏ Example:

```
int[] numbers = {1,2,3,4};
Response.Write(numbers.Length);
```

❏ Advantages of Arrays

- Stores **multiple values in one variable**.
- **Easy access using index**.
- Helps in **loop processing**.
- Saves **memory and coding time**.

❏ Disadvantages

- **Fixed size** (cannot change after creation).
- Can store **only same data type**.

Basics Controls**Q.-12. What are controls? Explain Types of controls in asp.net.**

- ❏ ASP.NET Web controls are objects on ASP.NET Web pages that run when the page is requested and that render markup to a browser.
- ❏ An array is a container that holds multiple values of the same type in a single variable.

Array in ASP.NET Using C#

- ❏ ASP.NET uses C# arrays the same way as normal C# applications because ASP.NET is built on the .NET framework.
- ❏ Arrays are commonly used in:
 - ❏ Storing temporary data
 - ❏ Binding data to controls (like GridView, DropDownList)
 - ❏ Handling multiple values (like form inputs)

Declaring and Initializing an Array in ASP.NET (C#).**Types of controls in asp.net****User Controls:**

- ❏ A user control is a kind of composite control that works much like an ASP.NET Web page—you can add existing Web server controls and markup to a user control, and define properties and methods for the control. You can then embed them in ASP.NET Web pages, where they act as a unit.

Web Server Controls:

- ❏ ASP.NET Web server controls are objects on ASP.NET Web pages that run when the page is requested and that render markup to the browser. Many Web server controls are similar to familiar HTML elements, such as buttons and text boxes. Other controls encompass complex behavior, such as a calendar controls, and controls that you can use to connect to data sources and display data.

Web Part Controls:

- ❏ ASP.NET Web Parts controls are an integrated set of controls for creating Web sites that enable end users to modify the content, appearance, and behavior of Web pages directly in a browser. The topics in this section provide information on what Web Parts are, how they work, and how to use them to create user-customizable ASP.NET Web pages.

Types of asp.net framework controls:

- ❏ The ASP.NET Framework contains over 70 controls. These controls can be spread in different types, these types are below:

Standard Controls

- ❑ The standard controls enable you to render standard form elements such as buttons, input fields, and labels. We examine these controls in detail in the following chapter, "Using the Standard Controls."

Validation Controls

- ❑ The validation controls enable you to validate form data before you submit the data to the server. For example, you can use a RequiredFieldValidator control to check whether a user entered a value for a required input field.

Rich Controls

- ❑ The rich controls enable you to render things such as calendars, file upload buttons, rotating banner advertisements, and multi-step wizards.

Data Controls

- ❑ The data controls enable you to work with data such as database data. For example, you can use these controls to submit new records to a database table or display a list of database records.

Navigation Controls

- ❑ The navigation controls enable you to display standard navigation elements such as menus, tree views, and bread crumb trails.

Login Controls

- ❑ The login controls enable you to display login, change password, and registration forms.

HTML Controls

- ❑ The HTML controls enable you to convert any HTML tag into a server- side control.

Basic Web Server Controls**1) The Label Server Control**

- ❑ The Label server control is used to display text in the browser.
- ❑ Because this is a server control, it is possible to dynamically change the text from your server-side code. It has Text Property which is used to assign caption to text.
- ❑ Label control can also have hot keys which is also known as access keys which are used to set focus in particular control like text box or button.
- ❑ Hot keys are assigned with the AccessKey attribute. In this case, Label1 uses a, and Label2 uses b. The second new attribute for the Label control is the AssociatedControlID attribute.
- ❑ The String value placed here associates the Label control with another server control on the form. The value must be one of the other server controls on the form. If not, the page gives you an error when invoked.
- ❑ See the following example of label control

```
<asp:Label ID="Label1" runat="server" Text="Name" AccessKey="a"
AssociatedControlID="TextBox1"></asp:Label><br />
<asp:TextBox ID="TextBox1" runat="server"></asp:TextBox><br />
<asp:Label ID="Label2" runat="server" Text="address" AccessKey="b"
AssociatedControlID="TextBox2"></asp:Label><br />
<asp:TextBox ID="TextBox2" runat="server"></asp:TextBox>
```

2) The TextBox Server Control

- ❑ The TextBox server control is one of the most used controls in web forms.
- ❑ Text box on the form is used by the end user to input text. You can map the TextBox control to three different HTML elements used in your forms.
 - ❑ Simple TextBox
 - ❑ Password TextBox
 - ❑ Multiline TextBox
- ❑ First, the TextBox control can be used as a standard HTML text box, as shown in the following code

```
<asp:TextBox ID="TextBox1" Runat="server"></asp:TextBox>
```

- ❑ Second, the TextBox control can allow end users to input their passwords into a form as shown in following code

```
<asp:TextBox ID="TextBox1" Runat="server" TextMode="Password"></asp:TextBox>
```

- ❑ Third, the TextBox server control can be used as a multiline text box as shown in the following code.

```
asp:TextBox ID="TextBox1" Runat="server" TextMode="MultiLine"  
Width="300px" Height="150px"></asp:TextBox>
```

3) The Button Server Control

- ❑ The Button control is used to display a push button.
- ❑ The push button may be a submit button or a command button. By default, this control is a submit button.
- ❑ A submit button does not have a command name and it posts the page back to the server when it is clicked.
- ❑ It is possible to write an event handler to control the actions performed when the submit button is clicked.
- ❑ A command button has a command name and allows you to create multiple Button controls on a page.
- ❑ It is possible to write an event handler to control the actions performed when the command button is clicked.

The CausesValidation Property

- ❑ If you have more than one button on your Web page and you are working with the validation server controls, you don't want to fire the validation for each button on the form. Setting the Causes Validation property to False is a way to use a button that will not fire the validation process.

The CommandName Property

- ❑ To have multiple buttons on your form all working from a single event, Command Name property is used.
- ❑ You must construct your Button controls in the manner explained below.

```
<asp:Button ID="Button1" Runat="server" Text="Button 1"  
OnCommand="Button_Command" CommandName="DoSomething1" />  
<asp:Button ID="Button2" Runat="server" Text="Button 2"  
OnCommand="Button_Command" CommandName="DoSomething2" />
```

- ❑ Looking at these two instances of the Button control, you should pay attention to following things.
- ❑ The first thing to notice is the OnCommand event, which points to an event called Button_Command.
- ❑ You can see that both Button controls are working from the same event so to make a difference between two event, value placed in the CommandName property.

3.1) The LinkButton Server Control

- ❑ The LinkButton server control is a variation of the Button control.

- ❑ It is basically the same except that the LinkButton control looks like of a hyperlink. But, it isn't a typical hyperlink. When the end user clicks the link, it behaves like a button.
- ❑ This is an ideal control to use if you have a large number of buttons on your Web form.
- ❑ A LinkButton server control is constructed as follows:

```
<asp:LinkButton ID="LinkButton1" Runat="server" OnClick="LinkButton1_Click">  
Submit your name to our database </asp:LinkButton>
```

3.2) The ImageButton Server Control

- ❑ The ImageButton control is also one type of Button control.
- ❑ It is almost exactly the same as the Button control except that it can show any custom image as the form's button instead of the typical buttons used on most forms.
- ❑ This means that one can create your own buttons as images and the end users can click the images to submit form data. A typical ImageButton is constructed as follows:

```
<asp:ImageButton ID="ImageButton1" Runat="server"  
OnClick="ImageButton1_Click" ImageUrl="MyButton.jpg" />
```

- ❑ The ImageButton control specifies the location of the image used by using the ImageUrl property.

4) The RadioButton Server Control

- ❑ RadioButton Server control enables user to select single item from multiple items.
- ❑ The RadioButton control allows you to place single or multiple Radio Button on a form.
- ❑ All the Radio Buttons are mutually works together if GroupName property is set to same value for all radio buttons.
- ❑ See the following example of radio button control.

```
<asp:RadioButton ID="RadioButton1" runat="server" Text="Yes" GroupName="Set1" />  
<asp:RadioButton ID="RadioButton2" runat="server" Text="No" GroupName="Set1"/>  
The RadioButtonList Server Control
```

4.1) RadioButton List Control

- ❑ The RadioButtonList server is used to display a collection of radio buttons on a Web page.
- ❑ The RadioButtonList control is quite similar to the CheckBoxList and other list controls in that it allows you to iterate through to see what the user selected, to make counts, or to perform other actions.
- ❑ See the following example of radio button list control.

```
<asp:RadioButtonList ID="RadioButtonList1" runat="server">  
<asp:ListItem Selected="True">English</asp:ListItem>  
<asp:ListItem>Russian</asp:ListItem>  
<asp:ListItem>Finnish</asp:ListItem>  
<asp:ListItem>Swedish</asp:ListItem>  
</asp:RadioButtonList>
```

5) The CheckBox Server Control

- ❑ Check boxes on a Web form enables user to make multiple selections from a collection of items or specify a value of an item to be yes/no, on/off, or true/false.
- ❑ The CheckBox control allows you to place single or multiple check boxes on a form.
- ❑ All the Check Boxes are independent from each others.
- ❑ See the following example of checkbox control.

```
<asp:CheckBox ID="CheckBox1" Runat="server" Text="CheckBox Demo example"
```

```
TextAlign="Left" />
```

5.1) The CheckBoxList Server Control

- ❑ The CheckBoxList server control is quite similar to the CheckBox control, except that it allows working with a collection of items rather than a single item.
- ❑ CheckBoxList controls can be also bind with datasource like MsAccess, Oracle, MSSQL etc. See the following example of CheckBoxList control.

```
asp:CheckBoxList id="Check1"  
RepeatLayout="flow"  
runat="server">  
<asp:ListItem>Item 1</asp:ListItem>  
<asp:ListItem>Item 2</asp:ListItem>  
<asp:ListItem>Item 3</asp:ListItem>  
<asp:ListItem>Item 4</asp:ListItem>  
<asp:ListItem>Item 5</asp:ListItem>  
<asp:ListItem>Item 6</asp:ListItem>  
</asp:CheckBoxList>
```

6) The DropDownList Server Control

- ❑ The DropDownList server control is similar to HTML select box on Web page.
- ❑ It should be used when you have a large collection of items from which you want the end user to select a single item.
- ❑ select box generated by the DropDownList control displays a single item and allows the end user to make a selection from a larger list of items.
- ❑ Here's the code for DropDownList control:

```
<asp:DropDownList ID="DropDownList1" Runat="server">  
<asp:ListItem>Car</asp:ListItem>  
<asp:ListItem>Airplane</asp:ListItem>  
<asp:ListItem>Train</asp:ListItem>  
</asp:DropDownList>
```

6.1) The ListBox Server Control

- ❑ The ListBox server control much like the DropDownList control.
- ❑ It displays a collection of items.
- ❑ The ListBox control behaves differently from the DropDownList control in that it displays list of item to the end user at a time as well as end user can make multiple selections from the collection.
- ❑ A typical List Box control appears in code as follows:

```
<asp:ListBox ID="ListBox1" Runat="server">  
<asp:ListItem>Hematite</asp:ListItem>  
<asp:ListItem>Halite</asp:ListItem>  
<asp:ListItem>Limonite</asp:ListItem>  
<asp:ListItem>Magnetite</asp:ListItem>  
</asp:ListBox>
```

7) Other Basic Controls

7.1) The HyperLink Server Control

- ❑ The HyperLink server control is used to provide hyperlinks on your Web pages.
- ❑ Hyperlinks are used to transfer from one page to another.
- ❑ You can set the text of a hyperlink using the control's Text attribute:

```
<asp:HyperLink ID="HyperLink1" Runat="server" Text="Go to this page here"
NavigateUrl="~/Default2.aspx"></asp:HyperLink>
```

- ❑ This server control creates a hyperlink on your page with the text Go to this page here.
- ❑ When the link is clicked, the user is redirected to the value that is placed in the NavigateUrl property.
- ❑ It can also display Image instead of Text using ImageUrl property.
- ❑ It is specially used when one need to show dynamic Links based on user input or value of database.

7.2) Image Server Control

- ❑ The Image server control enables you to work with the images that appear on your Web page from the server-side code.
- ❑ Image server control has one important property, ImageUrl which is points to file & its location.
- ❑ It also has two other important property, height & width which are used to control height and width of the image. See the following example of image control.

```
<asp:Image ID="Image1" runat="server" ImageUrl="~/MyImage1.gif" />
```

7.3) Table Control

- ❑ The Table allows you to build an HTML table and specify its characteristics.
- ❑ A table can be built at design time with static content, but the Table control is often built programmatically with dynamic contents.
- ❑ Each Table control is made up of rows stored in the Rows collection of the control.
- ❑ Each row is made up of cells stored in the Cells collection of the each TableRow.
- ❑ You can display an image in the background of the Table control by setting the BackImageUrl property. By default, the horizontal alignment of the items in the table is not set.
- ❑ To add a single row to a Table control, you have to create new instances of the TableRow and TableCell objects.
- ❑ You create the TableCell objects first and then place them within a TableRow object that is added to a Table object.
- ❑ See the following example of table control.

```
<asp:Table ID="Table1" runat="server">
<asp:TableRowRunat="server" Font-Bold="True"
ForeColor="Black" BackColor="Silver">
<asp:TableHeaderCell>First Name</asp:TableHeaderCell>
<asp:TableHeaderCell>Last Name</asp:TableHeaderCell>
</asp:TableRow>
<asp:TableRow>
<asp:TableCell>one</asp:TableCell>
<asp:TableCell>two</asp:TableCell>
</asp:TableRow>
<asp:TableRow>
<asp:TableCell>three</asp:TableCell>
<asp:TableCell>four</asp:TableCell>
</asp:TableRow>
</asp:Table>
```

7.4) Panel Server Control

- ❑ The Panel server control used to hold a set of controls to process it or to lay out controls in asp.net web pages.

- ❑ It is basically a wrapper for other controls, which enable you to take a group of server controls along with other elements (such as HTML and images) and turn them into a single unit.
- ❑ The advantage of using the Panel control to hold a set of other control that can be processed as single unit.
- ❑ For example, setting the Font-Bold attribute to True causes each control or text within the Panel control to adopt this attribute, setting visible=false causes each control/text become invisible in panel.
- ❑ Panel control has the capability to scroll with scrollbars that appear automatically depending on the amount of information that Panel control holds.